You can put the loop inside a single it, which removes the extra metaprogramming but keeps the line-number problem. If a line of test code is executed multiple times, it's hard to see which iteration of the line caused failure.

My inclination would be to unroll the loop and write separate specs for each key/value pair. If you're concerned for some reason about how much space that takes up, it's worth mentioning that if the setup is that simple, you can write the code quite compactly:

```
describe "correctly determines threshold values" do
  specify { (User.new(role: :admin).threshold).to eq(5) }
  specify { (User.new(role: :super_admin).threshold).to eq(20) }
  specify { (User.new(role: :user).threshold).to eq(3) }
end
```

I think this is more clear and more likely to give useful information in the case of test failure.

I'll talk more about clarity issues throughout the book. In particular, the issue will come up when I discuss factories versus fixtures as ways of adding test data, in Chapter 6, *Adding Data to Tests*, on page 101.

Well Defined

A test is *well defined* if running the same test repeatedly gives the same result. If your tests are not well defined, the symptom will be intermittent, seemingly random test failures (sometimes called Heisenbugs, Heisenspecs, or Rando Calrissians).

Three classic causes of repeatability problems are time and date testing, random numbers, and third-party or Ajax calls. These all have the same root cause, which is that it is basically impossible to ensure that each test is run in a consistent environment. Dates and times have a nasty habit of never staying the same—the time is continually increasing, while random data stubbornly insists on being random. Similarly, tests that depend on a third-party service or even test code that makes Ajax calls back to your own application can vary from test run to test run, causing intermittent failures.

Dates and times tend to lead to intermittent failures when certain magic time boundaries are crossed. You can also get tests that fail at particular times of day or when run in certain time zones. Random numbers, in contrast, make it somewhat hard to test both the randomness of the number and that the arbitrary number is used properly in whatever calculation requires it.

The test plan is similar for dates, randomness, and external services—really, it applies to any constantly changing dataset. The key is to make the test